48

BÖLÜM: KUBERNETES OPERATORS – KENDI AKILLI ROBOTUNU KODLA | MODULE: KUBERNETES OPERATORS – BUILD YOUR OWN ROBOT SRE

Pratik Örnek: `SpringBootApp` CRD'sini Yöneten Mini Java Operator

Hands-on: Building a `SpringBootApp` Custom Operator in Java

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

Kendi canlı robotumuzu sahaya sürme vakti! Bir yazılımcı sadece 5 satırlık minicik bir `SpringBootApp` YAML'ı oluşturacak. Bizim Java operatörümüz bunu anında havada yakalayacak; arka planda otomatik olarak tam teşekküllü bir Deployment, bir Service ve bir Ingress oluşturup ayaklandıracak! Yazılımcı 100 satır karmaşık YAML yazmaktan kurtulacak!

💡 ELI5 Metaphor

🇬🇧 ENGLISH

Time to unleash our live robot onto the cluster! A developer submits an ultra-simple 5-line `SpringBootApp` YAML. Our Java Operator intercepts it instantly and automatically generates a full-blown Deployment, ClusterIP Service, and Ingress route under the hood! Developers are spared from writing 150 lines of boilerplate YAML forever!

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

Operatörün çalışma prensibi: 1) Kullanıcı `kubectl apply -f sample-springbootapp.yaml` ile CR oluşturur. 2) API Server watch eventi tetikler, JOSDK `reconcile` metodunu çağırır. 3) Operatör `client.apps().deployments().inNamespace(...)` ile Deployment'ın varlığını kontrol eder. 4) Yoksa `new DeploymentBuilder()...` akıcı (fluent) API'siyle Deployment oluşturur. 5) `ownerReference` ekleyerek 'sahiplik' kurar; böylece CR silindiğinde açılan tüm deployment ve servisler K8s Garbage Collector tarafından otomatik temizlenir!

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

Execution lifecycle: 1) User submits a `SpringBootApp` custom resource. 2) JOSDK dispatches a reconciliation cycle. 3) The reconciler checks for existing child resources via the Fabric8 fluent builder DSL. 4) If absent, it synthesizes matching Deployments and Services. 5) By setting `OwnerReferences`, the operator ensures that deleting the parent CR triggers automated garbage collection of all spawned resources.

☕ Java Spring Boot Pratiği

☕ Spring Boot in Action

```yaml
# Kullanıcının Yazdığı 5 Satırlık Rüya YAML:
apiVersion: apps.eli5.io/v1alpha1
kind: SpringBootApp
metadata:
  name: my-coffee-shop
  namespace: coffee-shop
spec:
  image: "ghcr.io/eli5-k8s/spring-boot-k8s-eli5:1.0.0"
  replicas: 3
  port: 8080
  envMessage: "Operator tarafından yönetilen harika kahve servisi!"

# Operatör bunu yakalar ve arkada Deployment + Service + Ingress'i OTOMATİK kurar!
```

⚠️ Acemi Tuzağı & Pro İpucu

⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

Alt kaynakları oluştururken mutlaka `OwnerReference` bağlamalısınız! Bağlamazsanız, kullanıcı ana `SpringBootApp` nesnesini sildiğinde arka plandaki Deployment'lar silinmez, kümede 'hayalet iş yükleri' (orphan resources) olarak kalır.

🇬🇧 WATCH OUT

Always attach `OwnerReference` to generated resources! Without owner references, child deployments become orphaned when the custom resource is deleted.

🎯 Özet Çıkarımlar

🎯 Key Takeaways

- Operatörler karmaşık çoklu kaynak üretimini tek bir basitleştirilmiş CRD'ye indirir.
- `OwnerReference` ile alt kaynakların yaşam döngüsü otomatik bağlanır ve temizlenir.
- Yazılımcı üretkenliği ve platform mühendisliği standartları tavan yapar.

- Operators condense sprawling multi-resource manifests into simple, elegant CRDs.
- OwnerReferences guarantee clean, cascade garbage collection upon deletion.
- Supercharges internal developer platforms (IDP) and engineering team velocity.